DATA DRIVEN SYSTEMS CHEAT SHEET

1. SPATIAL DATA MANAGEMENT

Spatial Data Characteristics

Spatial Data: Data with location (coordinates) and/or geometric extent (shape).
Vector approx: sequence of vertices (polygons, polylines). **Field:** raster/pixels.

Spatial Relationships (3 Types)

Topological: intersects, disjoint, contains, inside, adjacent, overlaps, equals
Key: intersects(o1,o2) = NOT disjoint(o1,o2)
Directional: north, south, east, west, relative positions
Distance: Euclidean distance, ranges (near, far)

Topological Definitions

disjoint: interior AND boundary don't intersect
intersects: interior OR boundary intersect
inside(o1,o2): interior(o1) subset of interior(o2)
contains(o1,o2): interior(o2) subset of interior(o1)
adjacent: interiors disjoint BUT boundaries intersect
overlaps: interiors intersect AND each has points outside other

Spatial Query Types

Selection/Range: Find objects satisfying predicate with query region
Nearest Neighbor (k-NN): Find k closest objects to reference point
Spatial Join: R join S = {(r,s): r in R, s in S, r theta s}

SDBMS Extensions (4)

1. Data Modeling: ADTs for spatial types (OGC standards)
2. Query Language: SQL + spatial functions (ST_Contains, ST_Length)
3. Storage/Indexing: MBRs, R-trees, filter-refine
4. Query Evaluation: cost/selectivity models

Filter-Refine Framework

Problem: Polygon intersection = O(n log n) where n = edges
Solution: 1) Filter with MBR (cheap) -> candidates; 2) Refine exact geometry
MBR: Minimum Bounding Rectangle enclosing object
Key insight: For most topological predicates, MBRs use 'intersects'

No Total Ordering Problem

Cannot sort multidimensional points preserving spatial proximity. No O(log n + k) guarantee like B-tree. Space-filling curves (Z-order, Hilbert) help but cannot guarantee proximity for arbitrary data.

Point Access Methods

Grid File: Divide space with hyperplanes into cells -> blocks. Good for uniform data, bad for skewed.
Space-filling curves: Map nD to 1D, index with B+-tree. Z-order: interleave bits of coordinates.
Quadtree: Tree, each non-leaf has 4 children (quadrants). Adaptive to data distribution.

R-tree Structure

Height-balanced tree (like B+-tree), each node = disk page
Entry format: (MBR, ptr). Leaf: MBR=object, ptr=id. Non-leaf: MBR=subtree extent
Constraints: [m, M] entries per node, m = 0.4M (40% full min)

R-tree Optimization Goals (4)

1. Minimize Area: reduce dead space, fewer false positives
2. Minimize Overlap: prevent multiple search paths (CRITICAL)
3. Minimize Margin: prefer square MBRs over stripes
4. Maximize Occupancy: fuller nodes = shorter paths

R*-tree Improvements

Insertion: Leaf level: min overlap increase. Non-leaf: min area enlargement.
Split: 1) For each dim, compute margin sum for all splits. 2) Choose dim with min margin. 3) Choose split minimizing overlap (ties: area).
Forced Reinsertion: On overflow, remove 30% farthest entries, reinsert. Improves occupancy.

Range Query Algorithm

For each entry e in node n:
  If e.MBR intersects query W:
    If leaf: add e.object to results
    Else: recurse into e.ptr

Object Clipping vs Overlapping (PAM → SAM)

Problem: Extended objects overlap multiple partitions
Solution 1 - Clipping: Split objects, distribute pieces (data replication)
Solution 2 - Overlapping (R-tree): Allow MBR overlap (no replication)
R-tree choice: more flexible, better balancing

2. MULTIDIMENSIONAL DATABASES & DW

Core Concepts

Cube: nD data structure. Cell = fact (non-empty). Measures = numerical values.
Dimension: Textual context for facts. Has a hierarchy of levels.
Granularity: Level of detail determined by dimension levels of facts.

OLAP vs OLTP

OLAP: Analytical, historical, read-heavy, denormalized, complex aggregating queries
OLTP: Transactional, current, write-heavy, normalized, simple targeted queries

FASMI (OLAP Requirements)

Fast response (seconds), **A**nalysis support, **S**hared multi-user, **M**ultidimensional hierarchies, **I**nformation management

OLAP Types

ROLAP: Relational DBMS. **MOLAP:** Array-based. **HOLAP:** Hybrid.

Fact Types

Event: Models events (sales). Unpredictable timing.
Snapshot: State at point in time (inventory). Predefined times.
Measureless: Event without measures (customer contact).

Measure Types

Additive: Sum along ANY dimension (sales count).
Semi-additive: NOT along Time (inventory level). The same item counted multiple times.
Non-additive: Cannot sum (averages, ratios).

Star Schema

1 dimension table per dimension (surrogate key, all levels, descriptions)
1 fact table (foreign keys to dims, measure columns)
Redundancy OK: dims = 1-5% storage, easier queries, no update problems

Snowflake Schema

Multiple tables per dimension (1 per level)
Normalized, less redundancy, harder to query (many joins)

Surrogate Keys Advantages

Better storage, prevents key re-use, better dim updates, efficient queries

Data Warehouse (Inmon - SITN)

Subject-oriented, **I**ntegrated (unified format), **T**ime-variant (historical), **N**on-volatile (no updates, only loads)

Kimball 4-Step Process

1. Choose Business Process (what to analyze)
2. Choose Granularity (level of detail)
3. Design Dimensions (context)
4. Choose Measures (numbers to track)

Analysis Operations

Roll-up: Go UP hierarchy, less detail, aggregate. **Drill-down:** Go DOWN, more detail.
Drill-out: Add dimension. **Drill-across:** Combine cubes via shared dims (full outer join).
Slice: Select specific dim value. **Dice:** Complex subsetting. **Pivot:** Rotate dimensions.

Conformed Dimensions/Measures

Conformed dims: identical OR one subset of other (values AND attributes)
Conformed measures: equivalently defined across cubes
Required to combine data from different cubes meaningfully

SQL Extensions

CUBE operator: GROUP BY CUBE(A,B) gives subtotals and grand total
Window functions: SUM(x) OVER (PARTITION BY y) - aggregate per row

MDX Basics

Produces pivot tables directly. Dimensions have hierarchies with levels.
Top level: (All). SELECT on COLUMNS/ROWS, FROM cube, WHERE = slicer

Multiple Hierarchies Example

Time dimension:
  Fiscal: Day→Month→FiscalQ→FiscalYear→T
  Calendar: Day→Month→CalendarQ→CalendarYear→T
Common lowest levels (Day, Month)
Different grouping above

Balanced Tree Requirement

1. Uniform height: all paths same length
2. No level jumping: only immediate parent-child
3. Single parent: each value has exactly one parent

Example violation: Washington DC (no state parent)

3. DELTA LAKE

Problem with Cloud Object Stores
Key-value stores, no multi-key atomicity, eventual consistency
Expensive LIST operations (1000 keys/call, tens-hundreds ms)
5-10ms base latency, 50-100 MB/s throughput
Need 8-10 concurrent reads to saturate 10 Gbps network

Existing Approaches Problems
Directories of files: No atomicity, eventual consistency, expensive LIST, no versioning
Custom engines (Snowflake): All I/O through metadata service, vendor lock-in, complexity

Delta Lake Solution
Transaction log stored IN object store (no separate servers)
Data in Parquet format (columnar, compressed)
ACID transactions via optimistic concurrency
Lakehouse = data lake costs + DW management features

Directory Structure
mytable/ -> data partitions (date=2020-01-01/*.parquet)
_delta_log/ -> 000001.json, 000002.json, checkpoints

Log Actions
add: Add file (stats: count, min/max, nulls). **remove:** Remove file (tombstone).
metaData: Schema/config change. **protocol:** Version update.
commitInfo: Audit (user, operation). **txn:** App transaction ID (exactly-once).
dataChange: false if only rearranging (compaction).

Checkpoints
Parquet files summarizing log up to point (every 10 commits default)
Removes redundant actions, keeps active adds, recent removes
_last_checkpoint file points to most recent (OK if stale)

Read Protocol (5 Steps)
1. Read _last_checkpoint -> checkpoint_id (tolerates stale)
2. LIST from checkpoint for .json/.parquet (use largest ID)
3. Reconstruct state: checkpoint + logs -> active objects
4. Filter objects using statistics (min/max)
5. Read relevant objects in parallel (retry if not visible)

Write Protocol (5 Steps)
1. Identify current version r
2. Read data at version r (if needed)
3. Write new data objects (GUIDs)
4. ATOMICALLY write log record r+1 (put-if-absent or rename)
5. Optional: write checkpoint, update _last_checkpoint

Atomic Log Creation by Provider
GCS, Azure Blob: atomic put-if-absent
HDFS, ADLS Gen2: atomic rename
S3: coordination service (Databricks) or Spark driver state

Transaction Properties
Writes: Serializable (total order by log record IDs)
Reads: Snapshot isolation by default
Limitation: Single-table transactions only
Write rate: Several per second (optimistic concurrency conflicts)

Time Travel
Query past versions: **SELECT * FROM t VERSION AS OF 42**
Or: **TIMESTAMP AS OF '2020-01-01'**
Enabled by immutable objects + log (MVCC)
Configurable retention period for physical deletion

Streaming Features
Write compaction: Small objects for low latency, background compacts (dataChange=false)
Exactly-once: txn action stores (appId, version) atomically with data
Log tailing: LIST from last ID, skip dataChange=false

Z-Ordering
Space-filling curve for multi-dimensional ordering
Interleaves bits from multiple dimensions
Creates locality in ALL specified dimensions
Works with min/max stats for data skipping
Example: 54% avg skipping vs 25% for single sort

Key Commands
OPTIMIZE table: Compact small files (default 1GB)
OPTIMIZE ZORDER BY (a,b): Z-order by columns
VACUUM table RETAIN 168 HOURS: Delete old files
DESCRIBE HISTORY: Audit log

Performance Numbers
Delta cached: 17s for 1M partitions vs Hive: >1hr for 10K
Z-ordering: 43-54% object skipping per dimension
Real-world: 93% data skipping with multi-attribute queries

Similar Systems
Apache Hudi: similar, single concurrent writer
Apache Iceberg: similar, different feature set

Use Cases
ETL with ACID, Data warehousing/BI, GDPR compliance (DELETE), ML reproducibility (time travel), Streaming pipelines, SIEM/security

Exactly-Once Streaming Details
Mechanism: Writer atomically commits data + txn(appId, version)
On restart: Check log for last committed version, replay from next offset
Prevents duplicates even after failures
Spark Structured Streaming uses this natively

UPSERT/DELETE/MERGE (GDPR critical!)
```
DELETE FROM table WHERE userId = 'X'
MERGE INTO target USING source ON key
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
```
Atomically replaces updated objects via new add/remove records
Concurrent readers see consistent snapshot

Schema Evolution/Enforcement
Transactional schema updates (metaData action)
Schema enforcement: validates new data matches table schema
Can read old Parquet files without rewriting (backward compatible)
Catches wrong-schema errors early

Retention & VACUUM
Configurable retention period per table
VACUUM deletes files older than retention (default 7 days)
Time travel requires files within retention
Physical deletion delayed until after retention

When NOT to Use Delta Lake
✗ Sub-second latency streaming (use Kafka)
✗ Very high write rate >100 txn/sec (conflicts)
✗ Multi-table transactions (not supported)
✗ Primary key lookups (use HBase/Kudu)

AUTO OPTIMIZE
Databricks feature: automatically compacts on writes
Set table property to enable
Background process, no manual OPTIMIZE needed

Consistency by Provider
S3: Read-after-write (exceptions), LIST eventual
GCS: Read-after-write, LIST strong, put-if-absent
Azure Blob: Read-after-write, LIST eventual, put-if-absent
ADLS Gen2: Strong consistency, atomic rename

Checkpoint Mechanics
Default: every 10 commits
Combines actions: keeps active adds, recent removes, latest metadata
Parquet format for efficient querying
Checkpoint failure = no corruption (log remains truth)

QUICK FORMULAS & MNEMONICS

Polygon intersection: O(n log n). R-tree node: [0.4M, M] entries
FASMI: Fast Analysis Shared Multidimensional Information
SITN: Subject Integrated Time-variant Non-volatile
Measures: Additive (all dims), Semi (not Time), Non (none)
R*-tree split: Margin->axis, Overlap->split point
Delta: Log in object store, Parquet data, optimistic concurrency

EXTRAS
R-tree Dead Space & Overlap
Dead space: Empty area in MBR not covered by objects
Overlap: MBRs at same level intersect
Both cause false positives in queries
R*-tree minimizes overlap (most critical)
Level Properties (Multidim)
Non-hierarchical info for level (e.g., city population)
Stored in dimension table as column
Data Mart vs Data Warehouse
DW: Enterprise-wide, multiple subjects, shared
DM: Subset of DW, single subject, department-specific
DM loaded FROM DW (already cleaned) OR provides logical view
Never: Private data marts per department (breaks integration)